

Manual of ICGLE template flow conversion Python tool

Attentions of the original runset

- Make sure the original runset is valid for ICV parser with no error. The tool also validates it with ICV at the beginning.
- Avoid using complicated pre-define (`#ifdefine`, `#elseif`) in your runset

 Recommended

```
#define d_FLAG

#ifdef d_FLAG
// active code
#else
// inactive code
#endif
```

 Not supported

```
#ifdef d_FLAG
// inactive code
#else
// active code
#endif

#define d_FLAG

#ifdef d_FLAG
// active code
#else
// inactive code
#endif
```

- Avoid using complicated command style binary operator in your runset. If your runset already contains that, please use the function call style instead.

 Recommended

```
C = and(A, B);
E = not(or(C, D), F);
K = M or N;
```

 Not supported

```
E = ((A and B) or D) not F;
O = M external2_edge [extension = NONE] N;
```

- Violation block is not supported in template flow and will be **DELETED** by the tool. If your runset is using violation block to output errors, please convert them into data.



```
oasis_err_list : list of write_error_map_s = { };
oasis_data_list : list of write_layer_map_s = { };

rRULE @= { @ "OPC_RULE";
           copy( gRULE );
}

oasis_err_list.push_back({ rRULE, { 11001 } });

oasis_file = oasis_library("./EXAMPLE.oas");
write_oasis(oasis_file, errors = oasis_err_list, layers =
oasis_data_list);
```



```
oasis_err_list : list of write_error_map_s = { };
oasis_data_list : list of write_layer_map_s = { };

oasis_data_list.push_back({ gRULE, { 11001 } });

oasis_file = oasis_library("./EXAMPLE.oas");
write_oasis(oasis_file, errors = oasis_err_list, layers =
oasis_data_list);
```

- Customer function called inside template block can't call another customer function inside (no support of nested call).



```
func_1 : function(input : polygon_layer) returning output :
polygon_layer
{
    output = func_2(input);
}

//template_begin

B = func_1(A);

//template_end
```



```
func_1 : function(input : polygon_layer) returning output :
polygon_layer
{
    output = external1(input, distance = <0.02, extension = NONE);
```

```
}  
  
//template_begin  
  
B = func_1(A);  
  
//template_end
```

What you need to do before running the tool

Mark the beginning & end of the template block by adding some comments

Original runset

```
// define
oasis_data_list : list of write_layer_map_s = { };

// input layer
aMAIN = assign({ { 1, 0 } });

// pre-processing before ICGLE commands
gMAIN = size(aMAIN, -0.01);
gFRAG = length_edge(gMAIN, >0);

// ICGLE commands
gSEG1 = corner_dissect(layer1 = gFRAG, main_layer = gMAIN, dissect =
{==0.10});
gSEG2 = max_dissect(layer1 = gSEG1, main_layer = gMAIN, max_length =
5);

rule_all : list of rule_constraint_s = { {space = [0.056, 0.057),
width = > 0.03, length = < 0.085 , displacement = - 0.011} };
rule_bias_value = set_displacement_by_rule(layer1 = gSEG2,
main_layer = gMAIN, rule = rule_all);
rule_bias_out = apply_displacement(layer1 = rule1_bias_value,
main_layer = gCLEAN_EXCLUDE_M0B);

// output layer
oasis_data_list.push_back({ rule_bias_out, { 1, 0 } });
oasis_file = oasis_library("./EXAMPLE.oas");
write_oasis(oasis_file, layers = oasis_data_list, error_cell_prefix
= "");
```

Add comment **//template_begin** and **//template_end** to mark the template block:

```
// define
oasis_data_list : list of write_layer_map_s = { };

// input layer
aMAIN = assign({ { 1, 0 } });

//template_begin
// pre-processing before ICGLE commands
gMAIN = size(aMAIN, -0.01);
gFRAG = length_edge(gMAIN, >0);

// ICGLE commands
gSEG1 = corner_dissect(layer1 = gFRAG, main_layer = gMAIN, dissect =
{==0.10});

...

rule_bias_out = apply_displacement(layer1 = rule1_bias_value,
main_layer = gCLEAN_EXCLUDE_M0B);
```

```

//template_end

// output layer
oasis_data_list.push_back({ rule_bias_out, { 1, 0 } });
oasis_file = oasis_library("./EXAMPLE.oas");
write_oasis(oasis_file, layers = oasis_data_list, error_cell_prefix
= "");

```

Attention

- Template block should only contain ICV commands that are tended to be run inside, the following statement are forbidden:
 - o Definition and operations related to **write_layer_map_s**, **write_error_map_s**
 - o Layer assignment
 - o ICV options (hierarchy_option, ...)
 - o Write OASIS and GDSII operations
 - o ...
- ICGLE commands (dissect, bias, mrc) should always be put inside the template block.
- The input layers of template block should be polygon layers, and the output layers of template block should be one of polygon layer, edge layer, error layer or marker layer.
- Don't put the **//template_begin** and **//template_end** in the middle of a pre-define pair (#ifdef or #if #endif)



```

//template_begin
#ifdef d_FLAG

...

gRESULT = or_list({gNEGATIVE, gPOSITIVE});

//template_end

oasis_data_list.push_back({ gRESULT, { 1, 0 } });

#endif

```

Please re-write it into



```

//template_begin
#ifdef d_FLAG

...

gRESULT = or_list({gNEGATIVE, gPOSITIVE});

```

```

#endif

//template_end

#ifdef d_FLAG

oasis_data_list.push_back({ gRESULT, { 1, 0 } });

#endif

```

Suggestion

- The range of template block is not immutable. You can adjust to reduce the input layers of the template block, which might achieve better performance.

Here is another example to add template block

```

// define
oasis_data_list : list of write_layer_map_s = { };

// input layer
aMAIN = assign({ { 1, 0 } });
aREF = assign({ { 2, 0 } });

// pre-processing before ICGLE commands
gREF_EDGE = coincident_edge(aMAIN, aREF);

//template_begin
gMAIN = size(aMAIN, -0.01);
gFRAG = length_edge(gMAIN, >0);

// ICGLE commands
gSEG1 = corner_dissect(layer1 = gFRAG, main_layer = gMAIN, dissect =
{==0.10});

...

rule_bias_out = apply_displacement(layer1 = rule1_bias_value,
main_layer = gCLEAN_EXCLUDE_M0B);

//template_end

// output layer
oasis_data_list.push_back({ rule_bias_out, { 1, 0 } });
oasis_data_list.push_back({ gREF_EDGE, { 2, 0 } });
oasis_file = oasis_library("./EXAMPLE.oas");
write_oasis(oasis_file, layers = oasis_data_list, error_cell_prefix
= "");

```

For New DP Flow

Now we have added the support for the transform of multiple files runset to New DP Flow. Please note that a folder named `child_run_includes/` will be generated, in which the files are for the `#include` of child runset(bin.rs). So please don't miss this folder if you need to copy the transformed runset to another location to run.

Attention

New DP Flow only support polygon layers as output for this stage, so this tool will detect edge layers and error layers from output lists and remove them. In order to achieve apple to apple comparison, this tool also removes them from the generated quasi flow.

What you need to do to use this tool to generate New DP Flow runset?

1. Please follow the instructions above to complete the runset check and the addition of the `//template_begin` and `//template_end` tag.
2. For New DP Flow, a new runset called bin.rs will be generated, which will be executed by the child icv. `bin.rs` is a runset that needs to be executed independently, so it needs to run with the required `#include` directives, options() and global variables. It is difficult for this tool to gather all of this information. So need your help to mark them manually. But don't worry, this information is usually placed in adjacent code blocks, so it's simple to tag this information. The only work you need to do is to add `//child_run_include_begin` and `//child_run_include_end` at the beginning and end of this code block respectively.

Suggestion

- In general, we can mark it by the following simple method: add `//child_run_include_begin` at the very beginning of the main runset file, and add `//child_run_include_end` before the first assign() function. Because almost everything we need is before the position of calling assign() function.

The following is an example to explain how to find the location to add the flags.

```
//child_run_include_begin
// Add above flag at the beginning of the file
// #include directives
#include <icv.rh>

// global variables
vREC_ADI_LIMIT1 : const double = 0.026;      /* !! T2T cut 26nm */
vREC_ADI_LIMIT2 : const double = 0.028;      /* !! T2T cut 26nm */

// options()
run_options(
uppercase = run_options_uppercase
);
compatibility_options(
drc = {
spacing = { orphan_edge_membership = NONE }
}
);
error_options(
error_limit_per_check    = ERROR_LIMIT_UNLIMITED,
```

```

report_empty_violations = true
);

// #include directives
#include "../Include/assign.rs"
#include "../Include/offgrid_skew_acute.rs"

//child_run_include_end
// Add above flag before the first assign() function.
aN_M0B_ORG      = assign({ { 4510 } });
aN_M0B_CELL_ORG = assign({ { 3510 } });
aM0B_MOPC       = assign({ { 1510 } });

```

3. If you have already used this tool to generate the runset of template flow, then you only need to change one parameter to generate the runset of New DP Flow. (If you haven't used this tool, please be patient and read the following chapters first.)

- a) I have added a bool typed parameter named `use_newdp` to function `generate_template_flow()` to control whether this tool generates New DP Flow or template flow. And the default value is False. So my changes will not affect the original behavior of this tool. If you want to get New DP Flow runset using this tool, you should set this parameter as True. As below:

```

import icgle

runset_parser = icgle.parser(input_runset_path = '../runset/Main.rs',
output_dir = '../converted_runset')

runset_parser.parse_template_input_layers()
runset_parser.remove_violation_block()
runset_parser.parse_template_output_layers()
runset_parser.bypass_no_operation_layer()
runset_parser.add_cell_level()
runset_parser.generate_template_flow(\
.....
    use_newdp = True)

```

For module `detect_main_layers`

If you need to run `detect_main_layers()` module, you also need to mark `//child_run_include_begin` and `//child_run_include_end` in the runset. Since the global variables are needed for tracing main layers.

Where to get the tool

Python_conversion_tool will be built into ICV package, you could copy the files under the following path to your own directory. And make sure your environment support running Python.

Path:

\$ICV_HOME_DIR/contrib/icgle/template_conversion

It includes:

icgle.py

icv_processing_mode.rs

run_example.py

trace.pl

How to use the tool

Here is the **run_example.py** script, please modify as needed and run it.

- Make sure python3 can be run on your machine.
- Make sure ICV can be launched under current directory

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import icgle

runset_parser = icgle.parser(input_runset_path = './runset/Main.rs',
output_dir = './converted_runset', require_define = ['d_FLAG']);

runset_parser.parse_template_input_layers()
runset_parser.remove_violation_block()
runset_parser.parse_template_output_layers()
runset_parser.bypass_no_operation_layer()
# runset_parser.detect_main_layers()

runset_parser.add_cell_level()
runset_parser.generate_template_flow(template_ambit = 0.06,
template_block_function_name = 'template_block')
```

The tool consists of several parts.

- **initialization** Mandatory. It will parse the input runset and get its dependency runsets as a list; remove the comments; find template block and put it into a separate runset file.
- **parse_template_input_layers** Required for final step
generate_template_flow. Parse and get the input layer list of the template block, then save the information into the json config file.
- **remove_violation_block** Optional for final step
generate_template_flow. Ignore it if you can make sure there is no violation block in your runset. Otherwise, please use it.
- **parse_template_output_layers** Required for final step
generate_template_flow. Parse and get the output layer list of the template block, then save the information into the json config file.
- **bypass_no_operation_layer** Optional for final step
generate_template_flow. Parse and remove go-through layers of the template block (by identifying copy() statement), then re-parse input and output layers for template block and update the json config file.
- **detect_main_layers** Optional for final step
generate_template_flow. Automatically detect the type of each template input layer (MAIN_CONTEXT or CONTEXT), and backfill it into the runset.
- **add_cell_level** Required for final step
generate_template_flow. In this step, all ICV commands inside template block, which contain parameter: **processing_mode** will be switched into **CELL_LEVEL** mode.
- **generate_template_flow** Generate the template version of the input runsets

Parameters of the initialization of **icgle.parser**:

- **input_runset_path** [Required] The path of original runset
- **output_dir** [Required] Directory to save the output runsets and configuration file after each step. The existing runsets and configuration file in that directory will be overwritten. If the directory doesn't exist, the tool will create it.

Please put input runset files in a separate folder.

DON'T set your **output_dir** under the **input_runset_path**



```
runset_parser = icgle.parser(input_runset_path = './Main.rs',  
output_dir = './converted_runset', require_define = ['d_FLAG']);
```

- **working_dir** [Optional] Directory to save the temporary files and ICV logs. If it is not specified, **<output_dir>/temp** will be used by default. If the directory already exists before running the tool, it will be cleared by the initialization of the tool.
- **require_define** [Optional] List to save the required defines (-D in ICV option). If your runset already set it like: **#define d_FLAG**, please still add it to the list. If your runset doesn't define any, please ignore the parameter.

```
#define d_FLAG  
  
#ifdef d_FLAG  
// active code  
#else  
// inactive code  
#endif
```

```
runset_parser = icgle.parser(input_runset_path = './runset/Main.rs',  
output_dir = './converted_runset', require_define = ['d_FLAG']);
```

Parameters of the **generate_template_flow**:

- **template_block_function_name**
[Optional] **String**. Specify the function name that wraps the template block commands. The default value is **template_block**
- **template_ambit**
[Optional] **Float number**. Ambit is the extension distance outward tar box to generate rec. It is calculated by the sum of max look-out distance and potential boundary extend distance. The unit is micrometer, default value is **0.06**.

Typically, the max look-out distance is the maximum of the length, width and space in rule table, and the settings in dissection.

- **template_enable_boundary_handle**
[Optional] *Boolean or Integer*. Specify whether the boundary handle is added in the output runset. The default value is **True**. If you enable boundary handle, the default post-processing type will be selected (the behavior is to use oversized-tar to cut output layers).
Another post_processing behavior is to use tar_box to cut output layers. You could choose this type by setting the parameter value to integer **3**
- **preprocessing_type**
[Optional] *Integer number*. Specify the type of pre-processing. The default value is **1**.
1: Using oversized-tar to cut input layers. input_layer = and(or(tar, rec), oversized_tar).
2: Simply get input layers via or(tar, rec).
- **template_boundary_extend**
[Optional] *Float number*. Specify the template boundary extension in template boundary handle, the unit is micrometer. The value should be positive, default value is **0.0**. If there is polygon grow behavior inside template block, the template boundary in boundary handle should be also grow. If **template_enable_boundary_handle** is **False**, this parameter is ignored.
- **template_hierarchy_is_flatten**
[Optional] *Boolean*. Specify whether the generated running script is in flatten mode or optimized hierarchy mode. The default value is **True**.
- **template_max_template_size**
[Optional] *Integer number*. Specify the max template size generated in running script. Unit is micrometer, the default value is **25**.
- **template_bin_number**
[Optional] *Integer number*. Specify the number of bin generated in running script. The default value is **20**.
- **template_reverse**
[Optional] *Boolean*. Specify whether the template reverse is added in the output runset. The default value is **False**.
- **host_disk**
[Optional] *String*. Specify the ICV option *-host_disk* generated in running script. The default value is **/tmp**. If it is None or empty string, the ICV option *-host_disk* is removed.
- **host_init**
[Optional] *Integer number*. Specify the ICV option *-host_init* generated in running script. The default value is **None**. If it is None, the ICV option *-host_init* is removed.
- **input_DB_name**

- **input_cell_name**

[Optional] *String*. Specify the ICV option *-i* generated in running script. The default value is **<input_DB_name>**. If it is None or empty string, the ICV option *-i* is removed.
- **use_newdp**

[Optional] *String*. Specify the ICV option *-c* generated in running script. The default value is **<input_cell_name>**. If it is None or empty string, the ICV option *-c* is removed.

[Optional] *Boolean*. Specify whether the *new DP template flow* or *normal template flow* is generated in runset and running script. The default value is **False**.

What happens to your runset after running the tool?

Imagine this is your runset:

Main.rs

```
// code before template block
gMAIN = ...;

//template_begin

gFRAG = length_edge(gMAIN, >0);
// code in template block
gRESULT = ...;

//template_end

oasis_data_list.push_back({ gRESULT, { 1, 0 } });
// code after template block
```

=>

Main_template.rs

```
#include <icv.rh>

template_block : function(
    template_input_polygon_layer : list of polygon_layer,
    template_output_polygon_layer : out list of polygon_layer,
    template_output_edge_layer : out list of edge_layer,
    template_output_error_layer : out list of error_layer,
    template_output_marker_layer : out list of marker_layer,
    processing_mode : processing_mode_e
) returning void
{

    template_output_polygon_layer = {};
    template_output_edge_layer = {};
    template_output_error_layer = {};
    template_output_marker_layer = {};

    gMAIN = template_input_polygon_layer[0];

    gFRAG = length_edge(gMAIN, >0);
    // code in template block (cell level version)
    gRESULT = ...;

    template_output_polygon_layer.push_back(gRESULT);
}
```

Main_template.json

```
{
```

```

    "template_input_layer": {
        "gMAIN": {
            "layer_type": "polygon_layer",
            "is_main_layer": true
        }
    },
    "template_output_layer": {
        "gRESULT": {
            "layer_type": "polygon_layer"
        }
    },
    "function_call_in_template": []
}

```

Main.rs (for template flow)

```

#include "icv_processing_mode.rs"

// code before template block
gMAIN = ...;

#include "Main_template.rs"

#ifdef RUN_TEMPLATE_FLOW

/*
    Insert code here including:
    > template setup
    > template bin
    > call template block code in template mode as:
    template_block(
        template_input_polygon_layer,
        template_output_polygon_layer,
        template_output_edge_layer,
        template_output_error_layer,
        template_output_marker_layer,
        CELL_LEVEL);
*/

#else

/*
    Directly call template block code in non-template mode as:
    template_block(
        template_input_polygon_layer,
        template_output_polygon_layer,
        template_output_edge_layer,
        template_output_error_layer,
        template_output_marker_layer,
        HIERARCHICAL);
*/

#endif

oasis_data_list.push_back({ gRESULT, { 1, 0 } });

```

```
// code after template block
```

Main.rs (for New DP Flow)

```
#include "icv_processing_mode.rs"

// code before template block
gMAIN = ...;

#include "Main_template.rs"

#ifdef RUN_TEMPLATE_FLOW

/*
  Insert code here including:
  > template setup
  > template_bin_child_run
*/

#else

/*
  Directly call template block code in non-template mode as:
  template_block(
    template_input_polygon_layer,
    template_output_polygon_layer,
    template_output_edge_layer,
    template_output_error_layer,
    template_output_marker_layer,
    HIERARCHICAL);
*/

#endif

oasis_data_list.push_back({ gRESULT, { 1, 0 } });
// code after template block
```

bin.rs (only for New DP Flow)

```
#include "icv_processing_mode.rs"

#include "Main_template.rs"

/*
  > insert code between child_run_include_begin and
  child_run_include_end
*/

rec0 : polygon_layer = assign({{0}});
tar0 : polygon_layer = assign({{1}});

template_input_polygon_layer : list of polygon_layer = {};
template_input_polygon_layer.push_back(or(rec0, tar0));

/*
```



```

> call template block code in template mode as:
    template_block(
        template_input_polygon_layer,
        template_output_polygon_layer,
        template_output_edge_layer,
        template_output_error_layer,
        template_output_marker_layer,
        CELL_LEVEL);
*/

result_l : list of polygon_layer = {};
for (i = 0 to template_output_polygon_layer.size() - 1)
{
    result = or_list_bin({ template_output_polygon_layer[i],
template_output_polygon_layer[i]}, verbose = true, index = i);
    result_l.push_back(result);
}

for (i = 0 to result_l.size() - 1)
{
    required_layer(result_l[i]);
}

```

Besides, a running script (different for flatten and opt_hier) will be generated, including runset options (in comments), envs and ICV options.

If **template_hierarchy_option** is **flatten**, script **runme_flatten.csh** will be generated.

If **template_hierarchy_option** is **opt_hier**, script **runme_opt_hier.csh** will be generated.

runme_flatten.csh

```

# Add some runset options, if your runset already has them, then
merge the parameters (if the parameters listed below already exists
in your runset, please overwrite it)
# hierarchy_options(arefs_to_srefs={"*"}, explode={"*"});
# * If your runset contains directional operation, please add the
following options as well:
# prototype_options(symmetry = ASYMMETRIC);

setenv _ICV_QCUT_MAX_NUMBER_OF_CUT_CELLS 0
setenv _ICV_MULTI_BIG FALSE
setenv _ICV_TEMPLATE_PROCESSING TRUE
setenv _ICV_TEMPLATE_MAX_TEMPLATE_SIZE 25 # can be adjusted with
customer runset to improve performance
setenv _ICV_TEMPLATE_BIN_NUMBER 20 # should be adjusted to match the
CPU#

icv -icgle -D RUN_TEMPLATE_FLOW -disable qwc-opt -i <input_DB_name>
-c <cell_name> Main.rs

```

runme_opt_hier.csh

```
# Add some runset options, if your runset already has them, then
merge the parameters (if the parameters listed below already exists
in your runset, please overwrite it)
# hierarchy_options(arefs_to_srefs={"*"});
# * If your runset contains directional operation, please add the
following options as well:
# prototype_options(symmetry = ASYMMETRIC);

setenv _ICV_SPACE_GRID_OVERRIDE 1000 # can be changed to adjust the
hierarchy to improve performance
setenv _ICV_HAO_EXP_AMBIT_AREA_PLACEMENT_RATIO 2 # can be changed to
adjust the hierarchy to improve performance
setenv _ICV_EXPLODE_OVERLAP_Q_PERCENT 25 # can be changed to adjust
the hierarchy to improve performance
setenv _ICV_EXPLODE_OVERLAP_AREA_PERCENT 25 # can be changed to
adjust the hierarchy to improve performance
setenv _ICV_EXPL_OVLPL_FLT_PLF_FACTOR 1 # can be changed to adjust
the hierarchy to improve performance
setenv _ICV_HAO_EXPLODE_DATA_CELL_LIMIT 100 # can be changed to
adjust the hierarchy to improve performance
setenv _ICV_HAO_POST_VCELL_EXPLODE_DATA_CELL_LIMIT 100 # can be
changed to adjust the hierarchy to improve performance

setenv _ICV_QCUT_MAX_NUMBER_OF_CUT_CELLS 0
setenv _ICV_MULTI_BIG FALSE
setenv _ICV_TEMPLATE_PROCESSING TRUE
setenv _ICV_TEMPLATE_MAX_TEMPLATE_SIZE 25 # can be adjusted with
customer runset to improve performance
setenv _ICV_TEMPLATE_BIN_NUMBER 20 # should be adjusted to match the
CPU#

icv -icgle -D RUN_TEMPLATE_FLOW -D _ICV_EXPLODE_OVERLAP_Q_PERCENT -
disable qwc-opt -i <input_DB_name> -c <cell_name> Main.rs
```

An Example for running the tool

Here is an example to illustrate how to create a normal template and a new dp template run. Generally, there three steps: 1) Add begin & end comments for template and new dp flow; 2) Create *.py script; 3) Run the *.py script.

1) Add begin & end comments for template and new dp flow

For the non-template runset:

test_original.rs

```
#include <icv.rh>

#define d_FLAG

library(
  cell = "dchip",
  format = OASIS,
  library_name = "./two_layers.oas"
);

L_layer1 = assign({{1,0}});
L_layer2 = assign({{2,0}});

#ifdef d_FLAG
L_layer = L_layer1 not L_layer2;
#else
L_layer = L_layer1 xor L_layer2;
#endif

out_lib = oasis_library("out.oas");
write_oasis(
  out_lib,
  layers = {
    {L_layer, {0, 0}}
  }
);
```

Create new file input_file/test.rs and insert the comments of template_begin, template_end, child_run_include_begin, child_run_include_end. This input_file/test.rs can be used for creating both normal template and new dp template run:

input_file/test.rs

```
//child_run_include_begin

#include <icv.rh>

#define d_FLAG

library(
  cell = "dchip",
  format = OASIS,
```

```

    library_name = "./two_layers.oas"
);

//child_run_include_end

L_layer1 = assign({{1,0}});
L_layer2 = assign({{2,0}});

//template_begin

#ifdef d_FLAG
L_layer = L_layer1 not L_layer2;
#else
L_layer = L_layer1 xor L_layer2;
#endif

//template_end

out_lib = oasis_library("out.oas");
write_oasis(
    out_lib,
    layers = {
        {L_layer, {0, 0}}
    }
);

```

2) Create *.py script

You could start with file **run_example.py** and make some modifications according to your needs.

Here give two examples: a) flatten normal template flow; b) hierarchical new dp template flow.

a) Create a flatten normal template case

For creating normal template case, you need to set **use_newdp = False** (or not set this option, its default value is False). For setting the hierarchy to be flatten, you need to set **template_hierarchy_is_flatten = True**, besides you need to manually add **hierarchy_options(arefs_to_srefs={"*"}, explode={"*"});** in the output runset, for this change, you can reference the [output_files/example_template/test.rs](#). The following is the example for *.py script:

template_conversion/run_example_template.py

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import icgle

runset_parser = icgle.parser(input_runset_path =
'../input_file/test.rs', output_dir =
'../output_files/example_template', require_define = ['d_FLAG'])
runset_parser.parse_template_input_layers()
runset_parser.remove_violation_block()

```

```

runset_parser.parse_template_output_layers()
runset_parser.bypass_no_operation_layer()
runset_parser.detect_main_layers()
runset_parser.add_cell_level()
runset_parser.generate_template_flow(\
    template_block_function_name = "template_block", \
    template_ambit = 0.2, \
    template_enable_boundary_handle = True, \
    preprocessing_type = 1, \
    template_boundary_extend = 0.1, \
    template_hierarchy_is_flatten = True, \
    template_max_template_size = 25, \
    template_bin_number = 20, \
    template_reverse = False, \
    host_disk = "/tmp", \
    host_init = 20, \
    input_DB_name = '../two_layers.oas', \
    input_cell_name = 'dchip', \
    use_newdp = False)

```

b) Create a hierarchical new dp template case

For creating new dp template case, you need to set **use_newdp = True**. For setting the hierarchy to be hierarchical, you need to set **template_hierarchy_is_flatten = False**. The following is the example for *.py script:

template_conversion/run_example_newdp.py

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import icgle

runset_parser = icgle.parser(input_runset_path =
    '../input_file/test.rs', output_dir =
    '../output_files/example_newdp', require_define = ['d_FLAG'])
runset_parser.parse_template_input_layers()
runset_parser.remove_violation_block()
runset_parser.parse_template_output_layers()
runset_parser.bypass_no_operation_layer()
runset_parser.detect_main_layers()
runset_parser.add_cell_level()
runset_parser.generate_template_flow(\
    template_block_function_name = "template_block", \
    template_ambit = 0.2, \
    template_enable_boundary_handle = False, \
    preprocessing_type = 2, \
    template_boundary_extend = 0.0, \
    template_hierarchy_is_flatten = False, \
    template_max_template_size = 25, \
    template_bin_number = 20, \
    template_reverse = True, \
    host_disk = "/tmp", \
    host_init = 20, \
    input_DB_name = '../two_layers.oas', \

```

```
input_cell_name = 'dchip', \  
use_newdp = True)
```

3) Run the *.py script

Before you run *.py script, you need to copy files: icgle.py, icv_processing_mode.rs, trace.pl from template_conversion file to your *.py folder.

a) Flatten normal template case output

output_files/example_template/test.rs

```
#include "icv_processing_mode.rs"  
  
#include <icv.rh>  
  
#define d_FLAG  
  
library(  
  cell = "dchip",  
  format = OASIS,  
  library_name = "../two_layers.oas"  
);  
  
hierarchy_options(arefs_to_srefs={"*"}, explode={"*"});  
  
L_layer1 = assign({{1,0}});  
L_layer2 = assign({{2,0}});  
  
#include "test_template.rs"  
  
#ifdef RUN_TEMPLATE_FLOW  
  
ts_id = 1;  
  
max_template_size: integer =  
  strtoui($ICV_TEMPLATE_MAX_TEMPLATE_SIZE);  
if (max_template_size <= 0)  
{  
  error("Invalid max template size");  
}  
ts_ambit : double = 0.2;  
ts_boundary_extend : double = 0.1;  
  
ts_input_1 : list of template_layer_s = {};  
ts_out_1 : list of template_layer_s = {};  
tb_out_1 : list of template_layer_s = {};  
  
num_template_input_layer = 2;  
ts_input : template_layer_s;  
ts_input.tar = empty_layer();  
ts_input.rec = empty_layer();  
ts_input.pp = empty_layer();  
ts_input.source = L_layer1;  
ts_input.type = MAIN_CONTEXT;
```

```

ts_input_1.push_back(ts_input);
ts_input.source = L_layer2;
ts_input.type = CONTEXT;
ts_input_1.push_back(ts_input);

.....

```

The highlighted red part is what you need to manually add. You also need to pay attention to the **ts_input.type**, and manually change the **MAIN_CONTEXT/CONTEXT**, as you expected.

b) Hierarchical new dp template case output

output_files/example_newdp/test.rs

```

#include "icv_processing_mode.rs"

#include <icv.rh>
resolution_options(working_resolution_factor = ONE_X);
#ifdef RUN_TEMPLATE_FLOW
hierarchy_options(arefs_to_srefs={"*"});
#endif

#define d_FLAG

library(
  cell = "dchip",
  format = OASIS,
  library_name = "../two_layers.oas"
);

L_layer1 = assign({{1,0}});
L_layer2 = assign({{2,0}});

#include "test_template.rs"

#ifdef RUN_TEMPLATE_FLOW

ts_id = 1;

max_template_size: integer =
strtoi($_ICV_TEMPLATE_MAX_TEMPLATE_SIZE);
if (max_template_size <= 0)
{
  error("Invalid max tempalte size");
}
ts_ambit : double = 0.2;

ts_input_1 : list of template_layer_s = {};
ts_out_1 : list of template_layer_s = {};
tb_out_1 : list of template_layer_s = {};
child_out : list of polygon_layer = {};

.....

```

Features in future (if needed before iGraph)

- [Finished] Support generating new DP flow
- Support complicated command style binary operator

$E = ((A \text{ and } B) \text{ or } D) \text{ not } F;$
--

- Support generating mandatory runset options, envs, ICV options automatically
 - Support converting violation blocks instead of removing them
-
- Support main layer selection
 - Support multiple template blocks (mixed flow)
 - Support generating code to handle template boundary automatically.